Spotify-style Qt App with Embedded QWebEngineView Architecture and Build Guide

For a YouTube-focused desktop prototype using PySide6, QWebEngineView, queue management, and Brave-style ad-blocking integration.

Item	Value
Document type	System architecture and implementation planning guide
Target build	Desktop app, YouTube-first prototype
Core stack	PySide6, Qt WebEngine, Python domain layer, blocker engine adapter
Deliverable focus	Guidance only, no production code

1. Project framing

Treat the application as a desktop media shell with an embedded browser. The app owns the user experience, queue, state, and navigation policy. The browser pane only renders and loads web content. The blocker engine only decides whether a request is allowed.

System goal

- Spotify-like desktop layout
- Embedded QWebEngineView for playback
- YouTube-first scope for version 1
- Queue, next, previous, repeat, and shuffle logic
- Request interception and blocking through a blocker engine adapter

Scope boundary

In scope	Out of scope for v1
Library UI, queue UI, browser pane, playback state, URL resolution, request interception, settings persistence	General-purpose browser replacement, tabbed browsing, full account sync, perfect Brave-equivalent cosmetics, installer polish

2. OOA: actors and use cases

Primary actors

Actor	Role
User	Browses the library, selects tracks, manages queue, controls playback
Embedded web page	Renders media content and emits network requests
Ad blocking engine	Evaluates each request and returns allow or block decisions
Local storage	Stores queue snapshots, settings, history, and optional playlists

Main use cases

1. Open the application and load the library.
2. Select a track from the list.
3. Resolve the selection to a YouTube playback URL.
4. Load the target URL in QWebEngineView.
5. Intercept outgoing requests.
6. Allow or block requests through the blocker engine.
7. Add items to queue and play next or previous.
8. Save and restore queue and settings.

Textual use-case map

```
[User]
  -> Browse Library
  -> Select Track
  -> Add to Queue
  -> Remove from Queue
  -> Skip / Previous
  -> Toggle Repeat / Shuffle
```

```
  -> Change Settings

[Track Resolver]
  -> Resolve selection to playback URL

[Browser Pane]
  -> Load playback URL

[AdBlock Engine]
  -> Evaluate every intercepted request

[Storage]
  -> Save queue, settings, and history
```

3. OOD: layered architecture

Use a layered design so the UI does not talk directly to request filtering or persistence code.

Layer	Responsibility
Presentation	Widgets, windows, view models, user interactions
Application	Controllers, workflow coordination, command handling
Domain	Track model, queue rules, playback state, blocker contract
Infrastructure	Qt WebEngine, blocker adapter, JSON or SQLite persistence, URL resolvers

High-level dependency rule

- Widgets emit intent.
- Controllers decide what to do.
- Services execute workflow.
- Domain models hold rules and state.
- Infrastructure adapts external systems to domain interfaces.

4. Exact file and folder structure

```
app/
├─ main.py
├─ bootstrap.py
├─ config/
│  ├─ settings.py
│  └─ constants.py
├─ presentation/
│  ├─ windows/
│  │  └─ main_window.py
│  ├─ widgets/
│  │  ├─ sidebar_widget.py
│  │  ├─ track_list_widget.py
│  │  ├─ queue_widget.py
│  │  ├─ now_playing_widget.py
│  │  ├─ browser_pane.py
│  │  └─ search_bar.py
│  └─ viewmodels/
```

```
│     ├─ track_list_vm.py
│     ├─ queue_vm.py
│     └─ playback_vm.py
├─ application/
│  ├─ controllers/
│  │  ├─ app_controller.py
│  │  ├─ playback_controller.py
│  │  ├─ queue_controller.py
│  │  └─ navigation_controller.py
│  ├─ services/
│  │  ├─ playback_service.py
│  │  ├─ track_resolution_service.py
│  │  ├─ session_service.py
│  │  └─ settings_service.py
│  └─ dto/
│     ├─ playback_request.py
│     ├─ request_context.py
│     └─ queue_command.py
├─ domain/
│  ├─ models/
│  │  ├─ track.py
│  │  ├─ playlist.py
│  │  ├─ queue_item.py
│  │  ├─ playback_state.py
│  │  ├─ browser_request.py
│  │  └─ block_decision.py
│  ├─ enums/
│  │  ├─ repeat_mode.py
│  │  ├─ player_status.py
│  │  ├─ block_action.py
│  │  ├─ request_resource_type.py
│  │  └─ source_type.py
│  └─ interfaces/
│     ├─ blocker_engine.py
│     ├─ track_repository.py
│     └─ settings_repository.py
├─ infrastructure/
│  ├─ webengine/
│  │  ├─ web_profile_manager.py
│  │  ├─ web_page.py
│  │  ├─ request_interceptor.py
│  │  ├─ script_injector.py
│  │  └─ browser_event_adapter.py
│  ├─ brave/
│  │  ├─ brave_block_engine_adapter.py
│  │  ├─ brave_bridge_contract.py
│  │  └─ resource_loader.py
│  ├─ persistence/
│  │  ├─ json_settings_repository.py
│  │  ├─ json_queue_repository.py
│  │  └─ sqlite_track_repository.py
│  └─ resolvers/
│     └─ youtube_url_resolver.py
└─ tests/
   ├─ domain/
```

```
├─ application/
└─ infrastructure/
```

5. Core classes and responsibilities

Class	Main responsibility	Should not own
MainWindow	Compose widgets and connect UI to controllers	Queue rules, request filtering, persistence
BrowserPane	Own QWebEngineView and expose browser actions in app language	Playlist logic, blocker rule parsing
TrackListWidget	Display tracks and emit play or queue intent	Playback workflow
QueueWidget	Show queue and allow reorder or remove actions	Track resolution
NowPlayingWidget	Show current item and emit play control intent	URL loading or state mutation
AppController	Startup orchestration and dependency wiring	UI rendering details
PlaybackController	Handle play requests and browser loading flow	Low-level request info
QueueController	Manage queue commands	UI layout
NavigationController	Apply navigation policy and browser restrictions	Queue rules
RequestInterceptor	Map raw web requests and ask blocker engine for decision	UI state
BraveBlockEngineAdapter	Adapt blocker interface to Brave-compatible engine	Qt view behavior

6. Data types and enums

Recommended types

Data	Type	Reason
IDs	str	Simple, stable, easy to persist
URLs	str	Validate at boundaries only
Duration	int	Store seconds, avoid parsing noise
Timestamps	datetime	Clear domain meaning
Flags	bool	Direct state checks
Fixed categories	Enum	Avoid magic strings
Collections	list[T], dict[str, T]	Predictable behavior and ordering
Optional values	T | None	Make missing state explicit

Enums to define early

Enum	Values
RepeatMode	OFF, ONE, ALL
PlayerStatus	IDLE, LOADING, PLAYING, PAUSED, ERROR
BlockAction	ALLOW, BLOCK, REDIRECT
RequestResourceType	Map Qt resource types into your own enum
SourceType	YOUTUBE, LOCAL, PLAYLIST_ENTRY,

Enum	Values
	EXTERNAL_URL
QueueOrigin	USER_CLICK, AUTONEXT, RESTORED_SESSION

7. OOP rules

9. Keep Qt types at the edges. Convert raw Qt data into your own DTOs as early as possible.
10. Depend on interfaces in the domain. Example: BlockerEngine, TrackRepository, SettingsRepository.
11. Use composition over inheritance except where Qt requires inheritance.
12. Make one class the owner of one responsibility. Do not let widgets mutate everything.
13. Keep request interception lean. The request path should not perform heavy work.

8. Flow logic in pseudocode

Play request

```
FUNCTION handle_play_request(track_id):
    track = track_repository.get_by_id(track_id)

    IF track is null:
        show error "Track not found"
        RETURN

    playback_url = track_resolution_service.resolve(track)

    IF playback_url is null:
        show error "No playable URL"
        RETURN

    playback_state.status = LOADING
    playback_state.current_track_id = track.id
    playback_state.current_url = playback_url

    browser_pane.load_playback_url(playback_url)
    now_playing_vm.set_loading(track)
```

Add to queue

```
FUNCTION add_to_queue(track_id):
    track = track_repository.get_by_id(track_id)

    IF track is null:
        RETURN

    queue_item = create_queue_item(track, origin=USER_CLICK)
    queue_service.enqueue(queue_item)
    queue_vm.refresh(queue_service.get_items())
```

Get next item

```
FUNCTION get_next_playable_item():
    IF queue_service.has_explicit_items():
        RETURN queue_service.pop_next()
```

```
    IF playback_state.repeat_mode == REPEAT_ONE AND playback_state.current_track_id exists:
        RETURN track_repository.get_by_id(playback_state.current_track_id)

    IF playback_state.shuffle_enabled:
        RETURN playlist_service.get_random_next(excluding=history)

    next_track = playlist_service.get_next_sequential(current_track_id)

    IF next_track exists:
        RETURN next_track

    IF playback_state.repeat_mode == REPEAT_ALL:
        RETURN playlist_service.get_first_track()

    RETURN null
```

Intercept request

```
FUNCTION intercept_request(raw_info):
    request = request_mapper.map(raw_info)

    IF request.url is empty:
        ALLOW
        RETURN

    decision = blocker_engine.evaluate(request)

    IF decision.action == BLOCK:
        raw_info.block(true)
        log_blocked_request(request, decision)
        RETURN

    IF decision.action == REDIRECT:
        apply_redirect_if_supported()
        RETURN

    raw_info.block(false)
```

YouTube-only navigation guard

```
FUNCTION should_allow_navigation(target_url):
    host = extract_host(target_url)

    allowed_hosts = [
        "youtube.com",
        "www.youtube.com",
        "m.youtube.com",
        "youtu.be",
        "googlevideo.com",
        "www.googlevideo.com"
    ]

    IF host in allowed_hosts:
        RETURN true

    RETURN false
```

9. Flowcharts and state maps

Playback flow

```
User clicks item
-> Resolve track by id
-> If not found: show error
-> Resolve playback URL
-> If invalid: show error
-> Set state to LOADING
-> Load URL in browser
-> Browser reports success or failure
-> Update now playing and player state
```

Request blocking flow

```
Browser emits request
-> Map raw request to BrowserRequest
-> Ask BlockerEngine for BlockDecision
-> If BLOCK: cancel request
-> If ALLOW: continue request
-> If REDIRECT: apply redirect if supported
```

Playback state machine

```
IDLE -> LOADING     on play request
LOADING -> PLAYING  on page ready or media start
LOADING -> ERROR    on load failure
PLAYING -> PAUSED   on pause
PAUSED -> PLAYING   on resume
PLAYING -> IDLE     on stop or end with no next
PLAYING -> LOADING  on next track
ERROR -> LOADING    on retry
```

Textual class diagram

```
MainWindow
 ├── SidebarWidget
 ├── TrackListWidget
 ├── QueueWidget
 ├── NowPlayingWidget
 ├── BrowserPane
 └── AppController

AppController
 ├── PlaybackController
 ├── QueueController
 └── NavigationController

BrowserPane
 ├── QWebEngineView
 ├── CustomWebPage
 └── WebProfileManager

WebProfileManager
 ├── QWebEngineProfile
 ├── RequestInterceptor
 └── ScriptInjector
```

```
RequestInterceptor --> BlockerEngine
BlockerEngine <|.. BraveBlockEngineAdapter

PlaybackService
 ├── TrackResolutionService
 ├── QueueService
 ├── PlaybackState
 └── TrackRepository
```

10. Module-by-module implementation guide

Use this order. Build each module until it works in isolation before wiring the next one.

Module 1. Project skeleton and configuration

Build tasks:

- Create the package layout and folder structure.
- Define constants, settings schema, and environment assumptions.
- Decide where user data, queue snapshots, and settings files will live.
- Set naming conventions for IDs, URLs, enums, and DTOs.

Done when:

- A runnable app shell opens without errors.
- Configuration objects load with defaults.

Module 2. Domain models and enums

Build tasks:

- Create Track, Playlist, QueueItem, PlaybackState, BrowserRequest, and BlockDecision.
- Define RepeatMode, PlayerStatus, BlockAction, RequestResourceType, SourceType, and QueueOrigin.
- Keep these free from Qt imports.

Done when:

- All domain objects can be instantiated and validated in tests.
- No Qt dependency leaks into the domain package.

Module 3. Domain interfaces

Build tasks:

- Define BlockerEngine, TrackRepository, and SettingsRepository.
- Write the contract first so infrastructure remains replaceable.
- Keep method signatures small and explicit.

Done when:

- Controllers and services can depend on interfaces only.

Module 4. Repositories and persistence

Build tasks:

- Implement JSON settings persistence first.
- Implement queue snapshot persistence next.
- Use SQLite for tracks only if the library justifies it. JSON is acceptable for early prototypes.

Done when:

- The app can save and restore settings and queue snapshots.

Module 5. Queue service

Build tasks:

- Implement enqueue, dequeue, reorder, clear, and next-item selection rules.
- Handle priority of explicit queue over playlist order.
- Implement repeat and shuffle behavior in one place only.

Done when:

- Queue behavior works in tests without any UI.

Module 6. Track resolution service

Build tasks:

- Define how a Track becomes a playback URL.
- Support YouTube-first logic only in version 1.
- Reject tracks that cannot resolve to a URL.

Done when:

- A selected track can become a valid playback URL or a controlled failure.

Module 7. Playback service

Build tasks:

- Coordinate current track, state transitions, and browser load requests.
- Keep this service as the owner of playback state mutation.
- Let the browser pane be an executor, not the policy owner.

Done when:

- Play, next, previous, and error transitions are deterministic.

Module 8. Presentation widgets

Build tasks:

- Build SidebarWidget, TrackListWidget, QueueWidget, NowPlayingWidget, BrowserPane, and SearchBar.
- Each widget should emit intent, not perform deep business logic.
- BrowserPane should expose app-level actions such as load_playback_url.

Done when:

- Widgets render correctly and emit clean signals.

Module 9. View models

Build tasks:

- Create TrackListViewModel, QueueViewModel, and PlaybackViewModel.
- Shape data for widgets so widgets do not read repositories directly.

Done when:

- Widgets can refresh from view-model state instead of raw domain objects.

Module 10. Controllers

Build tasks:

- Create AppController, PlaybackController, QueueController, and NavigationController.
- Connect widget signals to controller actions.
- Map controller results back into view models and browser actions.

Done when:

- A user click can trigger a controlled playback flow.

Module 11. Web profile and page behavior

Build tasks:

- Create a dedicated QWebEngineProfile.
- Decide cookie, cache, storage, and permission policy.
- Implement a custom QWebEnginePage for popup, navigation, and console handling.

Done when:

- The embedded browser behaves consistently across launches.

Module 12. Request interceptor

Build tasks:

- Implement the QWebEngineUrlRequestInterceptor subclass.
- Map raw request info into BrowserRequest DTOs.
- Keep the path fast. Log lightly.

Done when:

- Every request is visible to your domain-level blocker contract.

Module 13. Simple blocker engine

Build tasks:

- Create a temporary rule-based blocker with manual host and pattern checks.
- Use it to prove your request flow before integrating Brave-compatible logic.

Done when:

- You can observe allowed and blocked requests inside the embedded browser.

Module 14. Brave blocker adapter

Build tasks:

- Wrap the Brave-compatible engine behind BlockerEngine.
- Do not let Rust or native bridge details leak upward.
- Add a fallback path that allows requests if the engine fails unexpectedly.

Done when:

- You can swap from simple blocker to Brave blocker without changing UI or controllers.

Module 15. Navigation guard and YouTube-only policy

Build tasks:

- Restrict navigation to approved hosts in version 1.
- Handle redirects and external links deliberately.
- Reject unsupported hosts with a clear UX rule.

Done when:

- The app stays inside its designed content boundary.

Module 16. Session restore and recovery

Build tasks:

- Restore queue, playback state, and key settings on startup.
- Handle corrupted state files safely.
- Always prefer a clean reset over a broken startup.

Done when:

- The app can restart without losing basic context.

Module 17. Test suite

Build tasks:

- Write domain tests first, then controller tests, then infrastructure smoke tests.
- Test queue rules, playback transitions, URL resolution, and blocker engine swap behavior.

Done when:

- The app can evolve without breaking its core logic silently.

Module 18. UI polish and hardening

Build tasks:

- Add error surfaces, empty states, busy states, and minor UX refinements.
- Avoid styling before the architecture works.
- Polish only after state and flows are stable.

Done when:

- Version 1 feels coherent and usable, not merely functional.

11. Suggested build order

14. Domain models and enums
15. Interfaces
16. Persistence and repositories
17. Queue service
18. Track resolution service
19. Playback service
20. Presentation widgets
21. View models
22. Controllers
23. Web profile, page, and request interceptor
24. Simple blocker engine
25. Brave blocker adapter
26. Session restore, tests, and UI polish

12. Acceptance criteria for version 1

- The app opens reliably.
- The library and queue render correctly.
- Clicking a track resolves and loads a YouTube playback URL.
- Queue add, remove, next, and previous work.
- Repeat and shuffle work through one queue rule owner.
- The request interceptor sees browser requests.
- The blocker engine can allow or block requests.
- The app can save and restore basic state across launches.

13. Final design check

Before you build features, ask one question: if the Brave-compatible blocker is replaced with a fake blocker, does the rest of the application still work? If the answer is yes, your architecture is properly separated.